

For personal use only,
please do not
distribute on
non-UBC domains.

02a - Cost of Change: Coupling

CPSC310

Nick Bradley & Kyle Chin



Learning Goals

By the end of the lecture you should be able to:

- Understand the two different forms of dependencies that can be present between code.
- Identify coupling in existing code and evaluate cost of change based on Degree, Locality, and Strength

A feature request

Ticket FEAT-0000

Currently the Instructor class only supports one `name` field, but we want to be able to specify a first and/or last name.

https://github.com/ubccpsc310/lecture_code

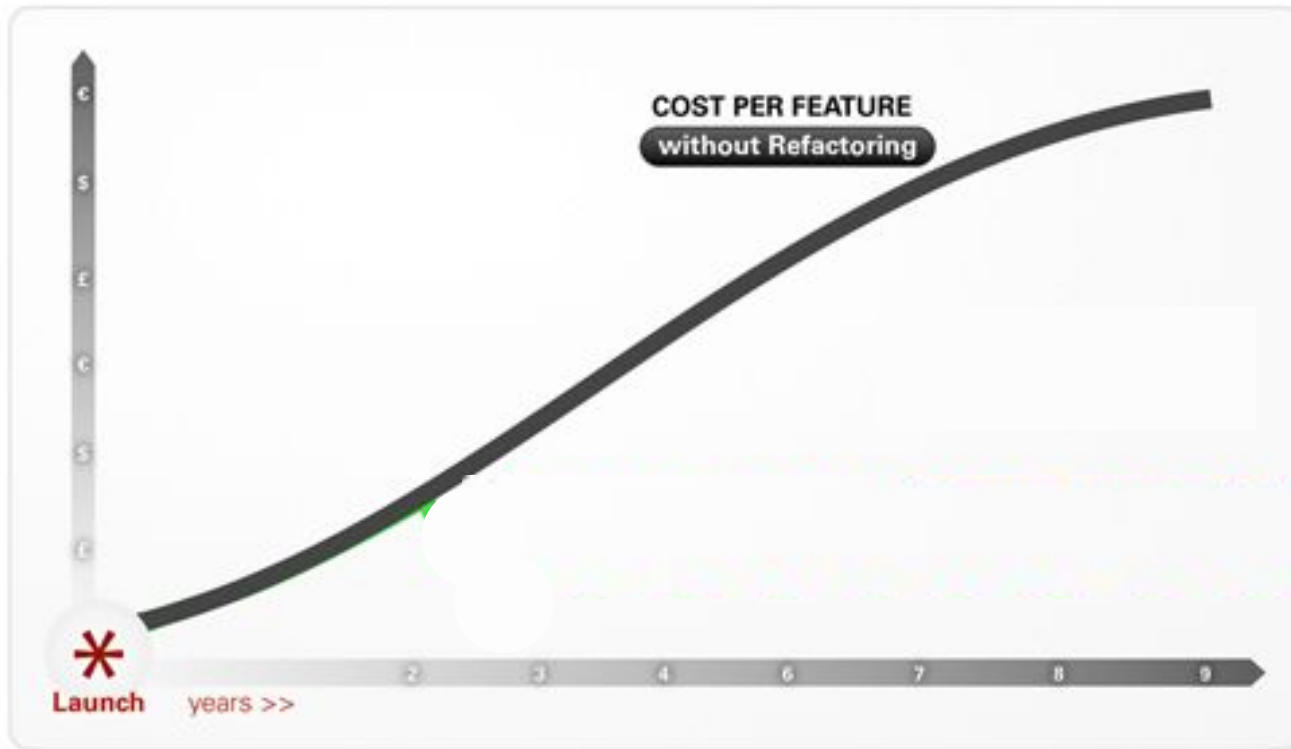
Observing the change

- Code is dependent on other code
- Changes in one code location may affect code in other locations too
- Dependencies can be
 - *Explicit*: static references that compiler sees
 - *Implicit*: references only visible dynamically at runtime
 - Requires **manually** locating all dependencies

A more serious feature request

Ticket FEAT-0001

A new measure was just passed giving seniors (residents 65+) free admission to Aquatics and Fitness, funded by an increase of the non-resident surcharge from 25% to 30%.





Dependencies in Code

How do we describe the dependencies between components?

- **Degree:** number of components affected by dependency
- **Locality:** how separate the dependent components are
- **Strength:** effort required to modify the dependency

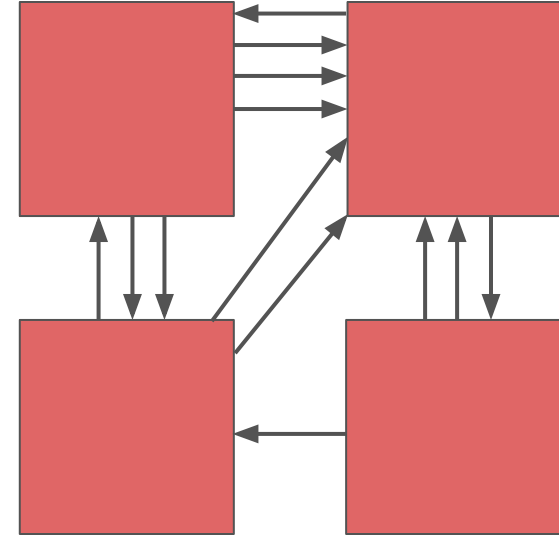
A.k.a.
Coupling!!

Coupling Intuition

Coupling is the relationship *between* components:

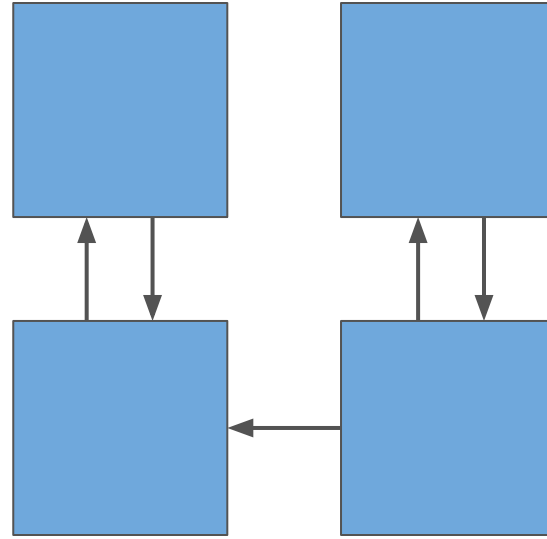
- It forms through dependencies—imports, method calls, shared data, and assumptions about other components.
- If we make a change in one component, what will the impact be on other components within the system?
- The type of dependency impacts the *risk* and *difficulty* of making changes.

Tight Coupling

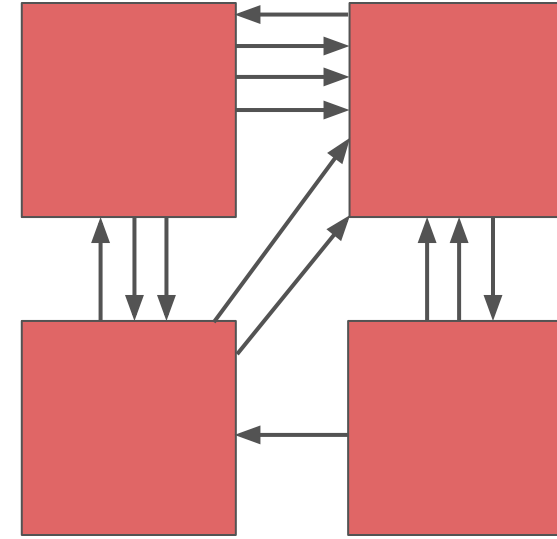


Tight
coupling

Loose Coupling is Cheaper

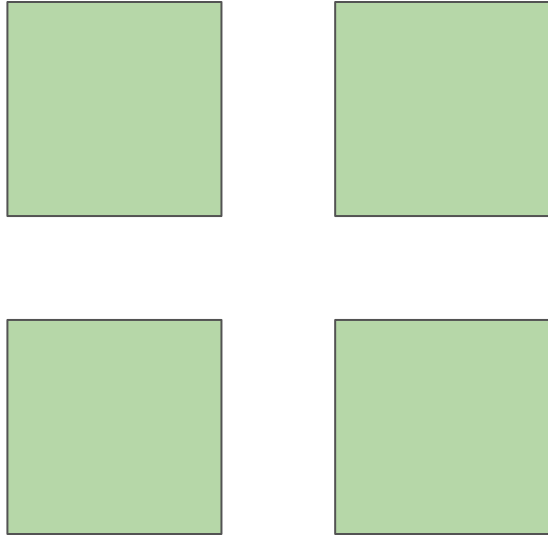


Loose
coupling

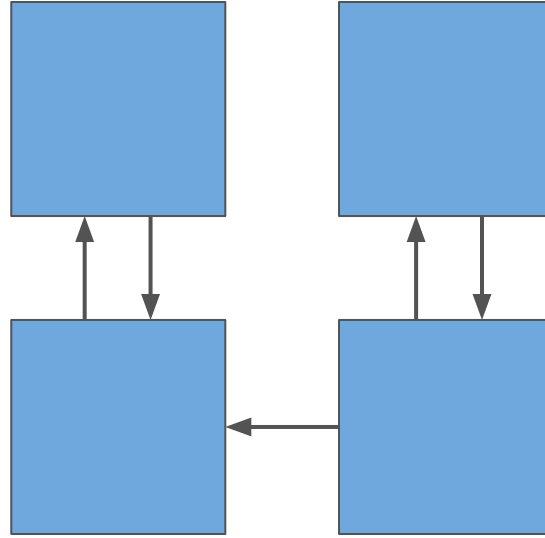


Tight
coupling

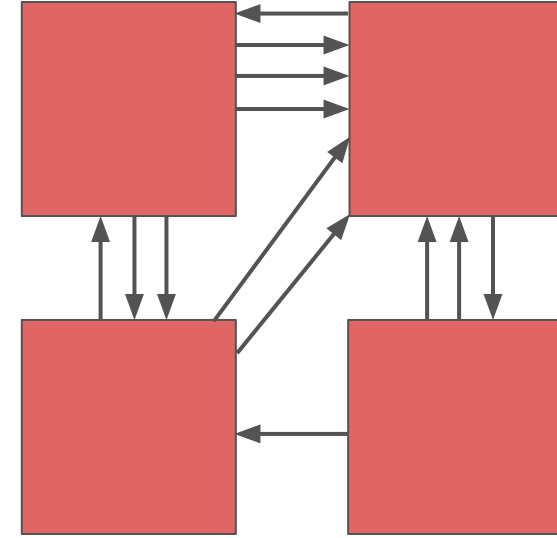
Some Coupling is Required



No
coupling



Loose
coupling



Tight
coupling

Coupling Strength (Connascence)

Measures how *strong* an edge is between two components.



Connascence Type	Multiple components must agree on...
Name	What something is called
Type	The shape or type of the data
Value	The meaning of a value
Position	An ordering of values
Algorithm	Computation following same steps

Learning Goals

By the end of the lecture you should be able to:

- Understand the two different forms of dependencies that can be present between code.
- Identify coupling in existing code and evaluate cost of change based on Degree, Locality, and Strength.

Cost of Change: Recap

- Software design have shortcomings that influence how much current and future changes cost
- Footprint and magnitude (reading and writing)
- Risk of missing part of the footprint

Coupling: Recap

A way of quantifying how the dependencies between software components affect the cost of change.

- **Degree** (how many things are related to the same concept)
- **Locality** (how far away are those things from each other)
- **Strength** (how tightly these things are connected)
 - A.k.a **Connascence**:
 - Name
 - Type
 - Value
 - Position
 - Algorithm
- Coupling can be either *explicit* or *implicit*